Kinematic Modelling and MATLAB-Based Analysis of the Dobot Magician Lite Robot Arm

Ezekiel, Olomije Moskovitch, Yosef Nastase, Radu Emilian
Oxley, John Raj, Shubham

Figure 1: Dobot Magician Lite, taken from [4].

ABSTRACT

This project presents the kinematic modelling and MATLAB-based analysis of the Dobot Magician Lite robot arm with a MATLAB implementation of the model.

1. INTRODUCTION

Robot manipulators are widely used across industrial, medical and educational domains because of their precision, repeatability and programmability [7]. Common applications include pick-and-place operations, assembly lines, and laboratory automation. In educational environments, robotic arms such as the Dobot Magician Lite are used for teaching robotics concepts [7].

The Dobot Magician Lite is a desktop serial manipulator, particularly suited for educational use, consisting of four revolute joints (J1–J4), forming an articulated robotic arm structure [7].

The aim of this project was to develop an analytical and MATLAB-based model of the Dobot Magician Lite and to evaluate its forward kinematics, inverse kinematics, Jacobian, singularities, dynamic equations, and validation behaviour.

2. KINEMATIC MODELLING

2.1 Robot configuration and assumptions

The kinematic model therefore treats the robot as a 4R serial manipulator, with θ_1 describing base rotation, θ_2 and θ_3 defining planar arm motion, and θ_4 controlling tool orientation.

2.1.1 The Passive Joint

The physical robot is a series of 4-bar linkages [1].

The passive joint P is introduced to maintain the constraint that the end-effector remains parallel to the ground. This is achieved by defining the passive joint angle as $-\theta_3 - \theta_2$.

The final rotation into the end effector frame α_3 happens at the end of the passive link, so the passive link's rotation still applies to the frame from joint 3 as shown in Table 1.

2.2 Denavit–Hartenberg Parameterisation

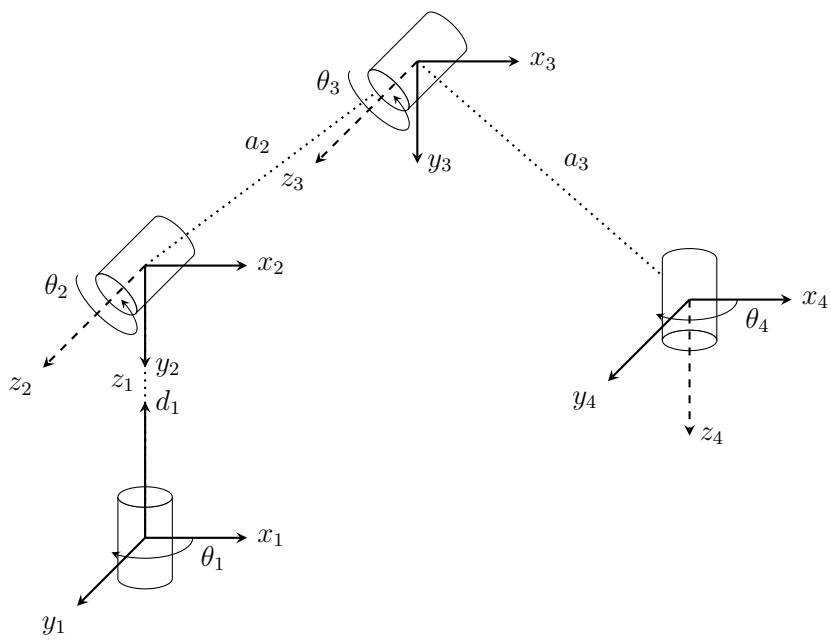

Figure 2: Denavit–Hartenberg frame assignment used for the kinematic modelling of the Dobot Magician Lite manipulator. z_2 and z_3 are coming out of the page.

The coordinate frames were assigned according to the standard Denavit–Hartenberg convention [7]. Each z-axis was aligned with the axis of rotation of the corresponding

joint, while each x-axis was defined along the common normal between adjacent joint axes.

The Denavit–Hartenberg parameter table derived from the frame assignment is

Link	a_i **(mm)**	α_i	d_i **(mm)**	θ_i
1 - Base	0	90°	150	θ_1
2 - Rear Arm	150	0°	0	θ_2
3 - Fore Arm	150	0°	0	θ_3
P - Passive	0	90°	0	$-\theta_3 - \theta_2$
4 - End effector	0	0°	0	θ_4

Table 1: Denavit–Hartenberg parameter table derived from the frame assignment.

2.3 Forward kinematics

The forward kinematics of the Dobot Magician Lite were derived using the Denavit–Hartenberg convention. Homogeneous transformation matrices were derived for each joint and multiplied to create the transformation matrix from the base frame to the end-effector frame, providing both rotation and cartesian postion [2, 3].

$$T_{04} = A_1 A_2 A_3 A_p A_4$$

2.4 Jacobian and singularity analysis

The Jacobian matrix relates joint rates to end-effector motion. In this work, the geometric Jacobian was used to describe the relationship between joint velocities and the linear and angular velocity of the end-effector, while an analytical form was considered for singularity analysis in terms of the selected task variables [7, 8]. Jacobian-based analysis is particularly useful for identifying configurations in which the manipulator loses mobility or becomes difficult to control.

Geometric Jacobian

The geometric Jacobian relates the joint velocities to the linear and angular velocity of the end effector:

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = J_g(q)\dot{q}$$

The geometric Jacobian is therefore:

$$J_g(q) = \begin{bmatrix} -r\sin(\theta_1) & \cos(\theta_1)(-L_2\sin(\theta_2) - L_3\sin(\theta_2+\theta_3)) & -L_3\sin(\theta_2+\theta_3)\cos(\theta_1) & 0 \\ r\cos(\theta_1) & \sin(\theta_1)(-L_2\sin(\theta_2) - L_3\sin(\theta_2+\theta_3)) & -L_3\sin(\theta_2+\theta_3)\sin(\theta_1) & 0 \\ 0 & L_2\cos(\theta_2) + L_3\cos(\theta_2+\theta_3) & L_3\cos(\theta_2+\theta_3) & 0 \\ 0 & -\sin(\theta_1) & -\sin(\theta_1) & 0 \\ 0 & \cos(\theta_1) & \cos(\theta_1) & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$

Singularities

A singularity occurs when the determinant of the analytical Jacobian becomes zero:

$$\det(J) = L_2 L_3 \sin(\theta_3) r = 0$$

This expression shows that singularities occur when either $\sin(\theta_3) = 0$ or $r = 0$. The condition $\sin(\theta_3) = 0$ corresponds to the arm being fully stretched or fully folded, so that the links become collinear. The condition $r = 0$ corresponds to the wrist lying on the base axis, where the manipulator loses motion authority in certain horizontal directions. Near these configurations, the velocity mapping becomes ill-conditioned and large joint velocities may be required to generate small end-effector motions [7, 8].

2.5 Inverse kinematics

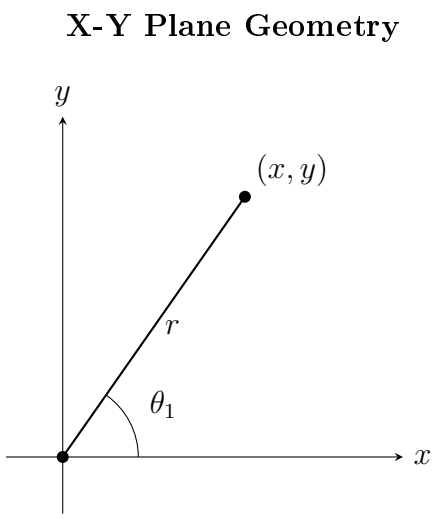

Figure 3: Top-view geometry used to compute the base joint angle θ_1.

The base joint angle is:

$$\theta_1 = \text{atan2}(y, x)$$

The remaining problem is reduced to a planar two-link geometry in the R–Z plane [2]. The function $\text{atan2}(y, x)$ is used because it returns the correct angular quadrant from the Cartesian coordinates [6].

R-Z Plane Geometry

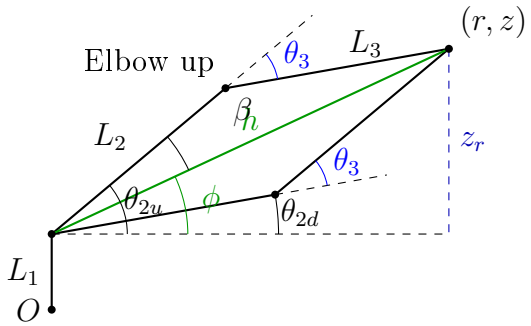

Figure 4: R-Z plane geometry used for inverse kinematics of the shoulder and elbow joints.

Two solution branches exist for reachable targets: an elbow-up configuration and an elbow-down configuration, giving shoulder angles of:

$$\theta_{2u} = \phi + \beta \quad \text{(for elbow-up)}$$

and

$$\theta_{2d} = \phi - \beta \quad \text{(for elbow-down)}$$

For θ_3, the forearm angle, we have two cases as well: **elbow-up** and **elbow-down**. However, the derivation for the inverse kinematic equation for this angle is a bit more complex.

Given

$$r^2 = x^2 + y^2 \tag{1}$$

$$z' = z - L_1 \tag{2}$$

$$r^2 = L_2 cos(\theta_2) + L_3 cos(\theta_2 + \theta_3) \tag{3}$$

$$z' = L_2 sin(\theta_2) + L_3 sin(\theta_2 + \theta_3) \tag{4}$$

Adding r^2 and z'^2 and simplifying gives us

$$\cos\theta_3 = \frac{r^2 + z'^2 - L_2^2 - L_3^2}{2L_2L_3} \tag{5}$$

$$\sin\theta_3 = \sqrt{1 - \cos^2\theta_3} \tag{6}$$

And the value of $\sin\theta_3$ above is negative for the elbow-up condition

$$\theta_{3d} = \text{atan2}(\sin\theta_3, \cos\theta_3) \quad \text{(for elbow-down)} \tag{7}$$

$$\theta_{3u} = \text{atan2}(-\sin\theta_3, \cos\theta_3) \quad \text{(for elbow-up)} \tag{8}$$

3. MATERIALS AND METHODS

3.1 Kinematic Modelling Approach

The kinematic model of the Dobot Magician Lite was developed using the Denavit–Hartenberg (DH) convention, which provides a systematic framework for describing the spatial relationship between adjacent links [7]. Each joint was modelled as a revolute joint, and the robot was approximated as an open kinematic chain, despite the physical presence of a four-bar linkage mechanism.

3.2 Inverse Kinematics Strategy

A geometric approach was selected for solving the inverse kinematics problem. This method was chosen due to the relatively simple structure of the robot and the availability of clear geometric relationships between joint angles and end-effector position.

This approach was preferred over numerical methods due to its computational efficiency, analytical clarity, and suitability for real-time applications [2, 5].

3.3 Dynamic Modelling Approach

The dynamic behaviour of the manipulator was considered using the standard rigid-body robot dynamics formulation [7, 8]

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q)$$

where τ is the vector of joint torques, $M(q)$ is the inertia matrix, $C(q, \dot{q})\dot{q}$ represents Coriolis and centrifugal effects, and $G(q)$ is the gravity vector.

In the present project, the dynamic equations were incorporated at a conceptual and modelling level to extend the kinematic model developed in MATLAB. However, a complete high-fidelity dynamic implementation was limited by the lack of detailed physical parameters such as link masses, mass centres, and inertia tensors from the manufacturer documentation [7]. Therefore, the work focused on establishing the dynamic form of the equations and explaining how such terms would be integrated into the MATLAB model for future extension.

3.4 MATLAB Implementation

The robot model was implemented as described in Section 2 as a MATLAB class with an API allowing for moving the joint angles, setting the end effect location in 3D space and viewing the transformation matrices, jacobian etc. See dobot.m for the full implementation.

A UI was built (see Figure 5) to allow users to interact with the model and unit tests $dobot_tests.m$ were added to ensure the model stayed accurate over time.

3.5 AI

AI tools, specifically Gemini & ChatGPT, were used to assist in generating MATLAB code for the UI and unit tests. The dobot.m MATLAB class was written by hand as that contains the mathematical model.

4. RESULTS AND VALIDATION

The MATLAB model was validated using forward-kinematics, inverse-kinematics and singularity test cases. Forward kinematics was checked by comparing computed end-effector positions with expected geometric outcomes, while inverse kinematics was verified by reconstructing joint configurations from target Cartesian positions. Additional tests were used to confirm singularity detection and rejection of unreachable targets.

Test case	**Input**	**Output**	**Remarks**
Forward kinematics	$q =$ (0.5236, 0.7854, 0, 0) rad	$(x, y, z) =$ (0.1837, 0.1061, 0.3191) m	Correct position
Inverse kinematics	Target (0.1837, 0.1061, 0.3191) m	$q =$ (0.5236, 0.7854, 0, 0) rad	Successfully reconstructed
Singularity test	$\theta_3 = 0$	$\det(J) = 0$	Correctly detected
Unit tests	Multiple valid/invalid cases	11/11 tests passed	Confirms consistency

Table 2: Validation of forward and inverse kinematics test results.

The validation results show good agreement between the analytical model and the MATLAB implementation. In particular, the consistency between the forward kinematics output and the inverse kinematics reconstruction confirms that the position mapping is internally correct. The singularity and unreachable-target tests further demonstrate that the implementation correctly handles important edge cases within the robot workspace.

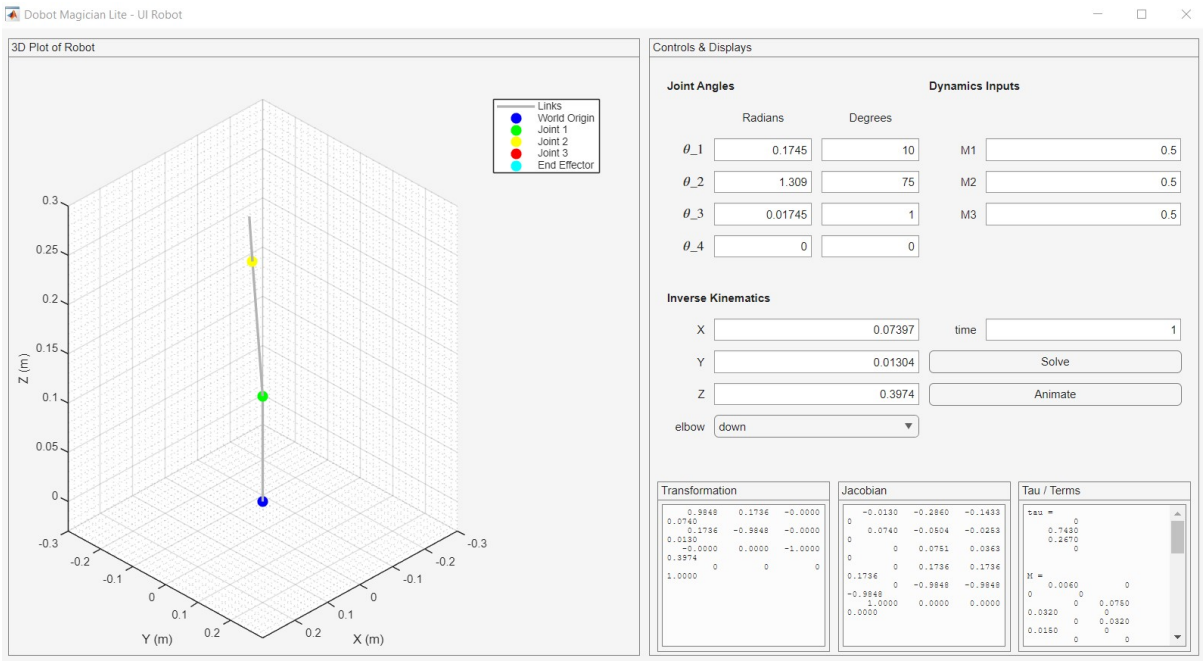

Figure 5: Successful inverse kinematics solution using the MATLAB-based UI.

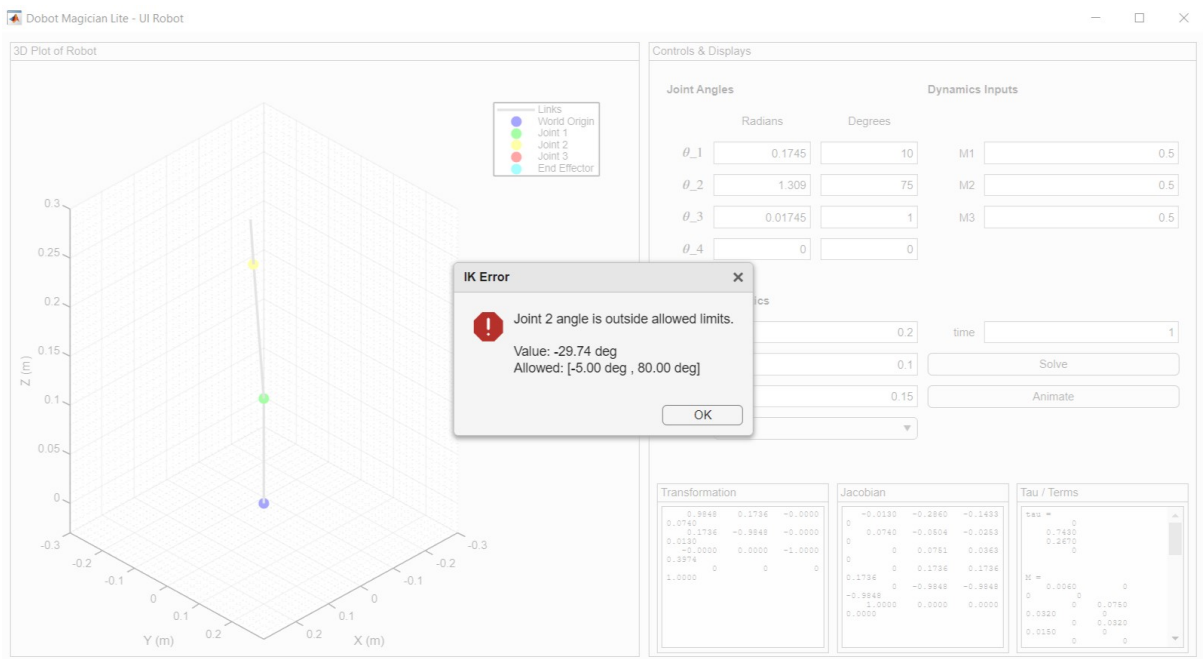

Figure 6: Detection of unreachable target in inverse kinematics.

The system correctly identifies when a target position lies outside the robot's reachable workspace. In this example, the computed distance h exceeds the maximum reach of the robot ($L_2 + L_3 = 0.30$ m), resulting in an error message. This demonstrates that the inverse kinematics implementation includes proper workspace validation and prevents physically invalid solutions.

5. DISCUSSION AND CONCLUSION

This project developed and validated a kinematic model of the Dobot Magician Lite using the Denavit–Hartenberg formulation and implemented the main equations in MATLAB. Forward and inverse kinematics were derived analytically, the Jacobian was obtained for differential motion analysis, and singular configurations were identified from the analytical model. The resulting MATLAB implementation was able to reproduce valid end-effector positions and reject targets lying outside the reachable workspace.

The actual robot is a series of 4-bar linkages [1]. After chosing to simplify the robot to a series of 4-bar linkages, the end effector was no longer parallel to the ground. The solution to this is detailed in Subsection 2.1.1.

A second challenge was the derivation of inverse kinematics in a form suitable for reliable computation; this was solved by reducing the problem to a planar geometric construction in the r–z plane.

As accurate link masses, centres of mass and inertia tensors were not available in sufficient detail from the manufacturer documentation, these were estimated in the model.

In the MATLAB UI, animation of the robot's arm, while using a cosine easing function, linearly changes the joint angles from the initial value to the final values. In the future, we would like to implement animation that used the Jacobian to make the robot follow a specific path.

6. REFERENCES

[1] Carnegie Mellon University. Dobot simulation resource, 2022. [Online]. Available: `https://courses.ideate.cmu.edu/16-375/f2022/ref/text/simulations/dobot.html`.

[2] J. J. Craig. *Introduction to Robotics: Mechanics and Control.* Pearson Education, Upper Saddle River, 3rd edition, 2005.

[3] J. Denavit and R. S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22:215–221, 1955.

[4] Dobot. *Dobot Magician Lite User Guide (DobotLab-based)*, n.d. [Online]. Available: `https://download.dobot.cc/product-manual/magician-lite/en/Dobot%20Magician%20Lite%20User%20Guide%20(DobotLab-based).pdf`.

[5] MathWorks. Derive and apply inverse kinematics to robot arm, 2023. [Online]. Available: `https://uk.mathworks.com/help/symbolic/derive-and-apply-inverse-kinematics-to-robot-arm.html`.

[6] MathWorks. Four-quadrant inverse tangent, n.d. [Online]. Available: `https://www.mathworks.com/help/matlab/ref/double.atan2.html`.

[7] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo. *Robotics: Modelling, Planning and Control.* Springer, London, 2009.

[8] M. W. Spong, S. Hutchinson, and M. Vidyasagar. *Robot Modeling and Control.* Wiley, Hoboken, 2006.